

INSTITUTO TECNOLOGICO SUPERIOR DE LERDO

**MATERIA: DESARROLLO DE APLICACIONES
MOVILES**

**ACTIVIDAD 4.2 CREACION DE UN MINI
REPRODUCTOR DE MUSICA EN FLUTTER**

ALUMNO:

DULCE MARIA REYES MARTINEZ

222310922

INGENIERÍA INFORMÁTICA

DOCENTE:

ING JESUS SALAS

Actividad 4.1

Creación de un Mini Reproductor de Música en Flutter

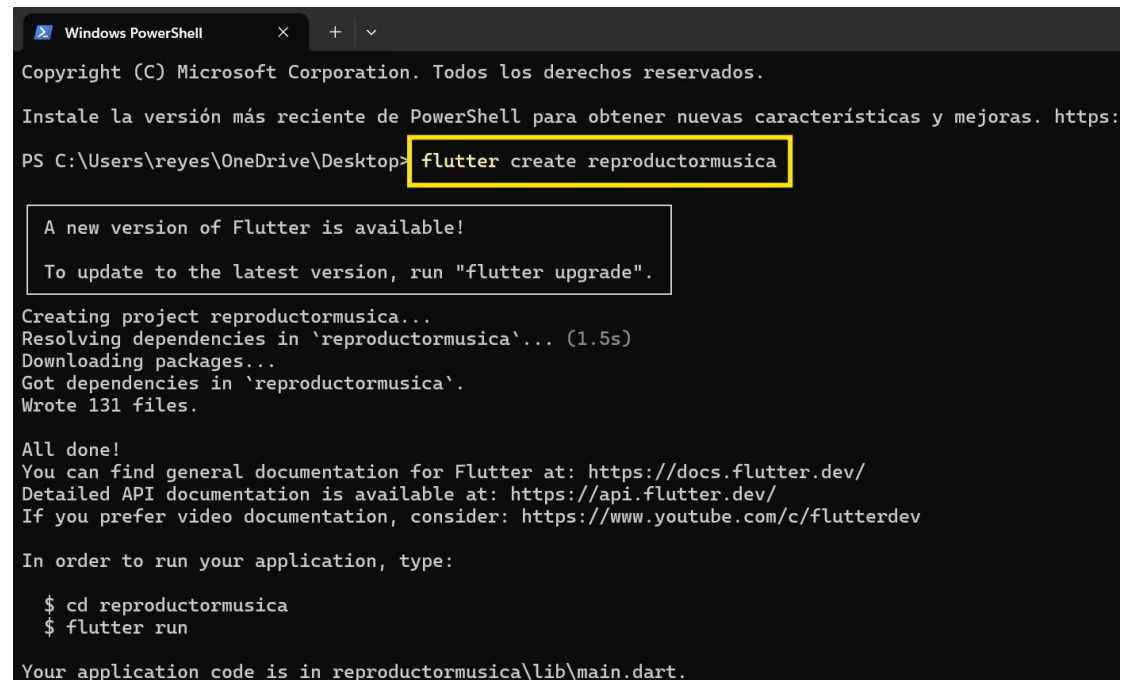
Introducción

En esta práctica se realizó una aplicación básica en Flutter que simula un reproductor de música. El objetivo fue aprender a utilizar Flutter y Dart para crear interfaces gráficas interactivas, agregar botones de reproducción y usar librerías externas para reproducir audio. También se trabajó con imágenes, canciones y personalización de la interfaz para hacer la aplicación más atractiva y funcional.

Desarrollo de la práctica

Paso 1. Creación del proyecto Flutter

Primero se creó un nuevo proyecto de Flutter utilizando Visual Studio Code y la terminal con el siguiente comando:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/powershell

PS C:\Users\reyes\OneDrive\Desktop> flutter create reproductormusica

A new version of Flutter is available!
To update to the latest version, run "flutter upgrade".

Creating project reproductormusica...
Resolving dependencies in 'reproductormusica'... (1.5s)
Downloading packages...
Got dependencies in 'reproductormusica'.
Wrote 131 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

In order to run your application, type:

$ cd reproductormusica
$ flutter run

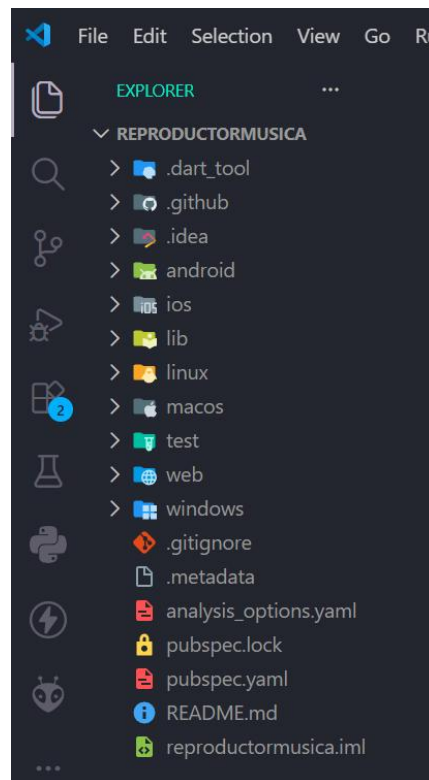
Your application code is in reproductormusica\lib\main.dart.
```

Después de crear el proyecto, es necesario ingresar a la carpeta generada desde la terminal. Para ello, se utiliza el siguiente comando:

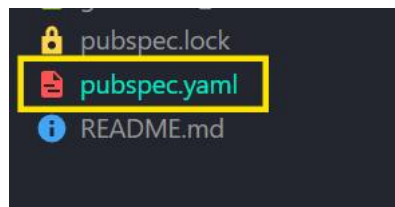
```
PS C:\Users\reyes\OneDrive\Desktop> cd reproductormusica
```

Una vez dentro de la carpeta del proyecto, se procede a abrirlo en Visual Studio Code con el siguiente comando:

```
PS C:\Users\reyes\OneDrive\Desktop\reproductormusica> code .|
```



En el archivo pubspec.yaml, dentro de la sección de dependencias, se deben agregar las dependencias necesarias para reproducir el audio



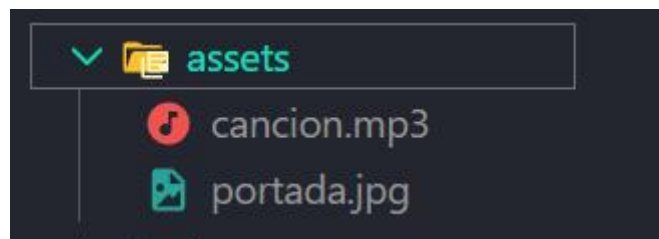
Paso 2. Configuración del archivo pubspec.yaml

Posteriormente se configuró el archivo pubspec.yaml, el cual sirve para agregar las dependencias y recursos necesarios del proyecto. En este archivo se añadieron las librerías just_audio y rxdart, necesarias para poder reproducir música dentro de la aplicación. También se agregó la carpeta assets, donde se almacenaron la canción y la imagen que se utilizaron en el reproductor. Una vez guardados los cambios, Flutter descargó automáticamente todas las dependencias necesarias para el funcionamiento de la aplicación.

```
pubspec.yaml
1  name: app2
2  description: "A new Flutter project."
3
4  publish_to: 'none'
5
6  version: 1.0.0+1
7
8  environment:
9    sdk: ^3.7.0
10
11  dependencies:
12    flutter:
13      sdk: flutter
14
15    just_audio: ^0.10.2
16    rxdart: ^0.28.0
17    cupertino_icons: ^1.0.8
18
19  dev_dependencies:
20    flutter_test:
21      sdk: flutter
22    flutter_lints: ^5.0.0
23
24  flutter:
25    uses-material-design: true
26
27    assets:
28      - assets/
29
```

Paso 3. Creación de la carpeta assets

Después de que se creó una carpeta llamada assets dentro del proyecto. En esta carpeta se agregaron los archivos multimedia que utilizaría la aplicación, como una imagen para representar la portada del álbum y una canción en formato MP3. Esto permitió que Flutter pudiera acceder fácilmente a los recursos utilizados en la interfaz y en la reproducción de audio.



Paso 4. Código principal de la aplicación

Explicación del código main.dart

```
1  import 'dart:ui';
2
3  import 'package:flutter/material.dart';
4  import 'package:just_audio/just_audio.dart';
5
```

En esta parte importé las librerías necesarias para el funcionamiento de la aplicación. material.dart me permite utilizar los widgets y crear toda la interfaz gráfica de Flutter. La librería just_audio.dart la utilicé para poder reproducir música y controlar el audio, mientras que dart:ui me sirve para agregar efectos visuales, como el desenfoque del fondo utilizando ImageFilter.blur().

```

6   void main() {
7       runApp(const MyApp());
8   }
9

```

Aquí se encuentra la función principal de la aplicación. La función `main()` es la primera que se ejecuta cuando inicia el programa. Dentro de ella utilicé `runApp()` para iniciar la aplicación y mostrar la clase `MyApp`, que contiene la estructura principal del reproductor.

```

10  class MyApp extends StatelessWidget {
11      const MyApp({super.key});
12
13      @override
14      Widget build(BuildContext context) {
15
16          return MaterialApp(
17              debugShowCheckedModeBanner: false,
18              title: 'Mini Reproductor',
19              theme: ThemeData.dark(),
20              home: const MusicPlayer(),
21          ); // MaterialApp
22      }
23  }

```

En esta parte creé la clase principal llamada `MyApp`, la cual hereda de `StatelessWidget` porque esta sección no necesita actualizarse dinámicamente. Dentro del método `build()` utilicé `MaterialApp` para configurar la aplicación. También desactivé la etiqueta de debug con `debugShowCheckedModeBanner: false`, agregué el título “Mini Reproductor”, activé el tema oscuro con `ThemeData.dark()` y definí que la pantalla principal será `MusicPlayer`.

```

25  class MusicPlayer extends StatefulWidget {
26    const MusicPlayer({super.key});
27
28    @override
29    State<MusicPlayer> createState() => _MusicPlayerState();
30  }

```

Aquí creé la clase `MusicPlayer`, que hereda de `StatefulWidget` porque la pantalla necesita actualizarse constantemente cuando cambia la canción, cuando el usuario pausa la música o cuando avanza el tiempo de reproducción. Flutter utiliza `_MusicPlayerState` para manejar toda la lógica y el estado del reproductor.

```

32  class _MusicPlayerState extends State<MusicPlayer> {
33
34    final AudioPlayer player = AudioPlayer();
35
36    int indiceCancion = 0;
37
38    Duration duration = Duration.zero;
39    Duration position = Duration.zero;
40
41    bool loadingSong = false;

```

En esta parte declaré las variables principales del reproductor. `AudioPlayer` es el objeto que se encarga de reproducir la música. La variable `indiceCancion` guarda la posición de la canción actual dentro de la lista. También utilicé `duration` para almacenar la duración total de la canción y `position` para saber en qué segundo va reproduciéndose. La variable `loadingSong` la utilicé para evitar errores cuando se cambia de canción muy rápido.

```

43     List<String> canciones = [
44         'assets/cancion.mp3',
45         'assets/cancion1.mp3',
46         'assets/cancion2.mp3',
47         'assets/cancion3.mp3',
48     ];
49

```

Aquí creé una lista llamada canciones donde almacené las rutas de los archivos MP3 que se encuentran dentro de la carpeta assets. Cada elemento de la lista representa una canción diferente.

```

50     List<String> portadas = [
51         'assets/portada.jpg',
52         'assets/portada1.jpg',
53         'assets/portada2.jpg',
54         'assets/portada3.jpg',
55     ];
56

```

En esta lista guardé las imágenes de portada de cada canción. Estas imágenes se utilizan tanto en el fondo dinámico como en la portada principal del reproductor.

```

57     List<String> nombresCanciones = [
58         'NORMAL (Explicit Ver.)',
59         'Sin Ti',
60         'Algo Mágico',
61         'Boy With Luv',
62     ];
63

```

Aquí almacené los nombres de las canciones que se mostrarán en pantalla.


```

64     List<String> artistas = [
65         'BTS',
66         'Morat, Jay Wheeler',
67         'Rauw Alejandro',
68         'BTS',
69     ];

```

En esta lista guardé los nombres de los artistas correspondientes a cada canción.

```

71     bool get isPlaying => player.playing;

```

En esta parte utilicé un getter llamado `isPlaying` para verificar si la música se está reproduciendo o no. Esto me ayuda a cambiar automáticamente el icono del botón entre play y pause.

```

73     @override
74     void initState() {
75         super.initState();
76
77         cargarCancion();
78     }

```

El método `initState()` se ejecuta automáticamente cuando la pantalla inicia. Aquí llamé a `cargarCancion()` para cargar la primera canción antes de comenzar la reproducción.

```

79     player.playerStateStream.listen((state) {
80
81         if (state.processingState == ProcessingState.completed) {
82             siguienteCancion();
83         }
84
85         setState(() {});
86     });

```

En esta parte agregué un listener para detectar cambios en el reproductor. Cuando una canción termina, automáticamente se ejecuta la función `siguienteCancion()` para pasar a la siguiente pista. También utilicé `setState()` para actualizar la interfaz gráfica.

```

88     player.durationStream.listen((d) {
89
90         if (d != null) {
91
92             setState(() {
93                 duration = d;
94             });
95         }
96     });

```

Aquí utilicé `durationStream` para obtener la duración total de la canción y guardarla en la variable `duration`.

```

98     player.positionStream.listen((p) {
99
100         setState(() {
101             position = p;
102         });
103     });
104 }

```

En esta parte utilicé `positionStream` para actualizar constantemente la posición actual de la canción mientras se reproduce.

```

106     Future<void> cargarCancion() async {
107
108         duration = Duration.zero;
109         position = Duration.zero;
110
111         await player.setAsset(
112             canciones[indiceCancion],
113         );
114     }
115

```

La función cargarCancion() la utilicé para cargar la canción seleccionada. Primero reinicié la duración y la posición a cero y después cargué el archivo MP3 utilizando player.setAsset().

```

116     @override
117     void dispose() {
118         player.dispose();
119         super.dispose();
120     }
121

```

En esta parte utilicé dispose() para liberar memoria cuando la aplicación se cierra. Esto ayuda a evitar problemas de rendimiento.

```

122     void playMusic() async {
123
124         await player.play();
125     }
126

```

La función playMusic() la utilicé para iniciar la reproducción de la canción actual.

```

127     void pauseMusic() async {
128
129         await player.pause();
130     }
131

```

La función `pauseMusic()` pausa la canción sin perder el tiempo actual de reproducción.

```

132     void replayMusic() async {
133
134         await player.seek(Duration.zero);
135
136         await player.play();
137     }

```

Aquí utilicé `replayMusic()` para reiniciar la canción desde el segundo cero y volver a reproducirla automáticamente.

```

139     void siguienteCancion() async {

```

Esta función la utilicé para cambiar automáticamente a la siguiente canción.

```

145         indiceCancion++;
146
147         if (indiceCancion >= canciones.length) {
148             indiceCancion = 0;
149         }
150

```

Aquí aumenté el índice de la canción actual. Si llega al final de la lista, automáticamente vuelve a la primera canción.

```

151     await player.stop();
152
153     await player.seek(Duration.zero);
154
155     await cargarCancion();
156
157     await player.play();
158

```

En esta parte primero detuve la canción actual, reinicié el tiempo, cargué la nueva canción y finalmente inicié la reproducción automática.

```

164     void anteriorCancion() async {
165

```

Esta función sirve para regresar a la canción anterior.

```

170         indiceCancion--;
171
172         if (indiceCancion < 0) {
173             indiceCancion = canciones.length - 1;
174         }
175

```

Aquí hice que si el índice baja de cero, automáticamente seleccione la última canción de la lista.

```

189   String formatDuration(Duration d) {
190
191       String minutes =
192           d.inMinutes.remainder(60).toString().padLeft(2, '0');
193
194       String seconds =
195           d.inSeconds.remainder(60).toString().padLeft(2, '0');
196
197       return "$minutes:$seconds";
198   }
199

```

Esta función la utilicé para convertir la duración de la canción a un formato más entendible como 02:35.

```

201   Widget build(BuildContext context) {
202

```

El método build() es donde construí toda la interfaz visual del reproductor.

```

203       return Scaffold(
204

```

Scaffold funciona como la estructura principal de la pantalla.

```

205         body: Stack(
206

```

Utilicé Stack para poder colocar elementos uno encima de otro, como el fondo, el desenfoque y el contenido principal.

```

209     AnimatedContainer(
210

```

Aquí utilicé `AnimatedContainer` para crear animaciones suaves cuando cambia la canción.

```

217         image: AssetImage(
218             portadas[indiceCancion],
219         ), // AssetImage
220

```

En esta parte cargué automáticamente la portada correspondiente a la canción actual.

```

226     BackdropFilter(
227
228         filter: ImageFilter.blur(
229             sigmaX: 30,
230             sigmaY: 30,
231         ), // ImageFilter.blur

```

Aquí apliqué un efecto borroso al fondo utilizando `ImageFilter.blur`.

```

237         gradient: LinearGradient(

```

En esta parte agregué un degradado oscuro para mejorar la visibilidad del texto y darle un diseño más moderno.

```

273 width: isPlaying ? 260 : 240,
274 height: isPlaying ? 260 : 240,
275

```

Aquí hice que la portada principal aumente ligeramente de tamaño cuando la música se está reproduciendo.

```

302 Text(
303
304 nombresCanciones[indiceCancion],
305

```

En esta parte mostré el nombre de la canción actual.

```

315 Text(
316
317 artistas[indiceCancion],
318

```

Aquí mostré el nombre del artista correspondiente.

```

335 Slider(
336

```

Utilicé un Slider para mostrar el progreso de la canción y permitir adelantar o retroceder el audio.

```

362 await player.seek(newPosition);
363

```

Aquí permití que el usuario cambie la posición de la canción moviendo el slider.


```

396     IconButton(
397
398         onPressed: anteriorCancion,
399

```

Botón para regresar a la canción anterior.

```

411     IconButton(
412
413         onPressed: replayMusic,
414

```

Botón para reiniciar la canción actual.

```

445         if (isPlaying) {
446
447             pauseMusic();
448
449         } else {
450
451             playMusic();
452         }

```

Aquí programé el botón principal para que pause o reproduzca la música dependiendo del estado actual.

```

457         isPlaying
458         ? Icons.pause
459         : Icons.play_arrow,
460

```

El icono cambia automáticamente entre pausa y reproducción.

```

470 IconButton(
471
472     onPressed: siguienteCancion,
473

```

Botón para avanzar a la siguiente canción.

```

528 child: ListView.builder(
529

```

Aquí utilicé `ListView.builder` para generar automáticamente la lista de canciones.

```

534 return GestureDetector(
535

```

Utilicé `GestureDetector` para detectar cuando el usuario toca una canción de la lista.

```

536 onTap: () async {
537

```

Cuando el usuario selecciona una canción, automáticamente se cambia el índice y se reproduce la nueva canción.

```

589 child: Image.asset(
590
591     portadas[index],
592
593     width: 60,
594     height: 60,
595

```

Aquí mostré la miniatura de cada portada dentro de la fila de reproducción.

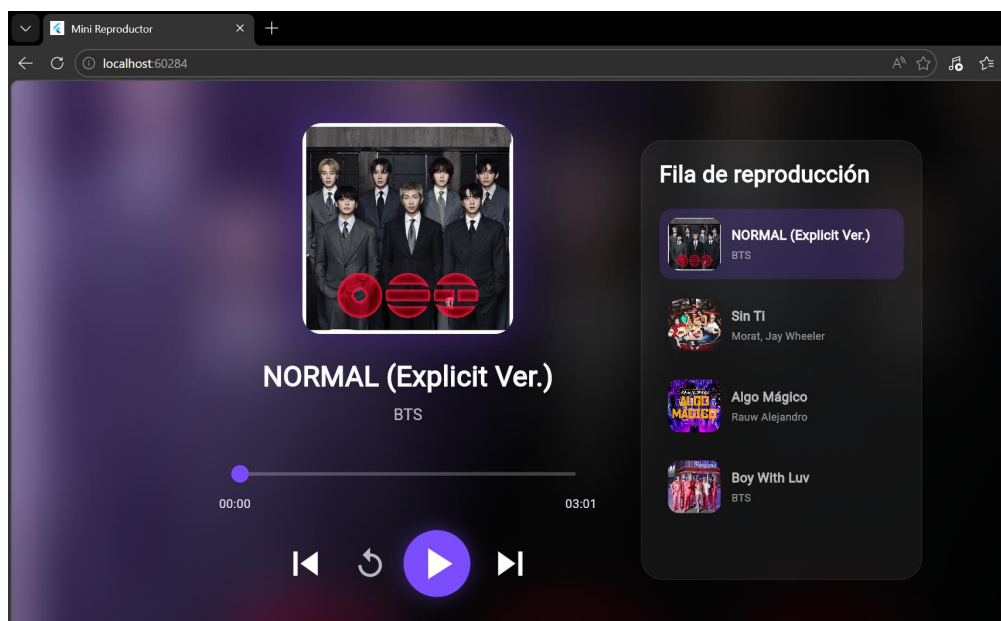
```

650     if (index == indiceCancion &&
651         isPlaying)
652     {
653         const Icon(
654             Icons.graphic_eq,
655             color: Colors.greenAccent,
656         ), // Icon

```

Finalmente, aquí agregué un icono verde que aparece únicamente cuando esa canción es la que se está reproduciendo actualmente.

Resultados



Al ejecutar la aplicación se obtuvo un reproductor de música completamente funcional. En la interfaz se muestra la portada de la canción seleccionada, junto con el nombre del tema y el artista correspondiente. También se observa un fondo dinámico con efecto de desenfoque que cambia automáticamente según la canción que se reproduce.

Además, se verificó el funcionamiento de los controles de reproducción, permitiendo reproducir, pausar, reiniciar, avanzar y retroceder entre canciones correctamente. La barra de progreso muestra el tiempo transcurrido y permite mover la reproducción a cualquier parte de la canción.

En la parte derecha se muestra la fila de reproducción, donde es posible visualizar todas las canciones disponibles y seleccionar cualquiera de ellas para reproducirla. Como se aprecia en la captura, la aplicación presenta una interfaz moderna y organizada, cumpliendo con los objetivos planteados para el desarrollo del reproductor de música.

PREGUNTAS DE REFLEXION

1. ¿Qué retos encontraste al seguir las instrucciones del video?

Uno de los principales retos que encontré fue comprender cómo se relacionaban algunos widgets entre sí para construir la interfaz completa. También tuve que prestar atención a la organización del código y a la configuración de los archivos de audio e imágenes dentro de la carpeta de assets para que la aplicación funcionara correctamente.

2. ¿Qué diferencia notaste entre una aplicación que simula una interfaz y una que realmente reproduce audio?

La principal diferencia es que una aplicación que solo simula una interfaz muestra los botones y elementos visuales, pero no realiza ninguna acción real. En cambio, una aplicación que realmente reproduce audio permite controlar la música, reproducir canciones, pausarlas, avanzar entre pistas y mostrar el progreso de reproducción en tiempo real.

3. ¿Qué widgets consideras más importantes o esenciales para construir una interfaz de un reproductor en Flutter?

Considero que los widgets más importantes son Scaffold, porque organiza la estructura principal de la pantalla; Container, para diseñar los elementos visuales; Text, para mostrar información de las canciones; Image, para las portadas; IconButton, para los controles de reproducción; Slider, para mostrar y controlar el avance de la canción; y ListView, para mostrar la lista de reproducción de manera ordenada.